**East West University**
**Department of Computer Science and Engineering**

# Course: CSE 475 (Machine Learning)
## Section - 03

| **Submitted To :** | **Submitted By :** |
|---|---|
| **Dr. Md. Tauhid Bin Iqbal**<br>Assistant Professor<br>Department of CSE<br>East West University | **Name** : Hafsa Ferdousi<br>**Student ID** : 2023-3-60-321<br>**Department** : CSE<br>**Date** : 03 /06 / 2026 |

Lab Report:01

### Create a Dataset with Data Quality Issues

A custom loan approval dataset containing 100 samples was created for this lab. The dataset includes various real-world data quality problems commonly found in machine learning datasets.

### Dataset Features

| | Customer_ID | Age | Monthly_Income | Credit_Score | Loan_Amount | \ |
|---|---|---|---|---|---|---|
| 0 | 1001 | 28.0 | 28278.0 | 690.0 | 178393.0 | |
| 1 | 1002 | 29.0 | 38434.0 | 829.0 | 95580.0 | |
| 2 | 1003 | 48.0 | 29165.0 | 565.0 | 99123.0 | |
| 3 | 1004 | 35.0 | 91237.0 | 563.0 | 344254.0 | |
| 4 | 1005 | 55.0 | 79987.0 | 662.0 | 285514.0 | |

| | Previous_Loans | Loan_Approved |
|---|---|---|
| 0 | 1.0 | No |
| 1 | 4.0 | No |
| 2 | 1.0 | No |
| 3 | 1.0 | No |
| 4 | 4.0 | No |

### Data Quality

| | Customer_ID | Age | Monthly_Income | Credit_Score | Loan_Amount | \ |
|---|---|---|---|---|---|---|
| count | 100.000000 | 90.000000 | 9.300000e+01 | 93.000000 | 9.200000e+01 | |
| mean | 1048.400000 | 38.455556 | 1.778300e+05 | 700.268817 | 1.349283e+06 | |
| std | 28.221705 | 17.838956 | 1.030045e+06 | 108.560356 | 1.039888e+07 | |
| min | 1001.000000 | -15.000000 | -1.000000e+04 | 200.000000 | -5.000000e+04 | |
| 25% | 1024.750000 | 28.000000 | 4.528900e+04 | 631.000000 | 1.582822e+05 | |
| 50% | 1047.500000 | 37.000000 | 6.830900e+04 | 709.000000 | 2.583455e+05 | |
| 75% | 1072.250000 | 48.000000 | 1.009140e+05 | 782.000000 | 3.637955e+05 | |
| max | 1098.000000 | 150.000000 | 9.999999e+06 | 999.000000 | 1.000000e+08 | |

| | Previous_Loans |
|---|---|
| count | 94.000000 |
| mean | 2.351064 |
| std | 2.593240 |
| min | -1.000000 |
| 25% | 1.000000 |
| 50% | 2.000000 |
| 75% | 4.000000 |
| max | 20.000000 |

```
    Customer_ID   Age  Monthly_Income  Credit_Score  Loan_Amount  \
0          1001  28.0         28278.0         690.0     178393.0
1          1002  29.0         38434.0         829.0      95580.0
2          1003  48.0         29165.0         565.0      99123.0
3          1004  35.0         91237.0         563.0     344254.0
4          1005  55.0         79987.0         662.0     285514.0
..          ...   ...             ...           ...          ...
95         1094   NaN             NaN         383.0          NaN
96         1095   NaN             NaN           NaN          NaN
97         1096   NaN         45627.0           NaN          NaN
98         1097   NaN             NaN           NaN          NaN
99         1098   NaN         77228.0         820.0          NaN

    Previous_Loans Loan_Approved
0              1.0            No
1              4.0            No
2              1.0            No
3              1.0            No
4              4.0            No
..             ...           ...
95             NaN            No
96             NaN            No
97             6.0            NO
98             NaN            NO
99             NaN           Yes

[100 rows x 7 columns]
```

**1. Missing Values**

Some records contain missing values where no data was entered.

Examples:

- Mis Age
- Missing Monthly Income
- Missing Loan Amount

Representation:

- Blank cells
- NaN values

```
Customer_ID         0
Age                10
Monthly_Income      7
Credit_Score        7
Loan_Amount         8
Previous_Loans      6
Loan_Approved       0
dtype: int64
```

| | Customer_ID | Age | Monthly_Income | Credit_Score | Loan_Amount | Previous_Loans | Loan_Approved |
|---|---|---|---|---|---|---|---|
| 80 | 1081 | NaN | 45000.0 | 720.0 | 150000.0 | 1.0 | Yes |
| 81 | 1082 | 35.0 | NaN | 680.0 | 120000.0 | 2.0 | No |
| 82 | 1083 | NaN | 55000.0 | 710.0 | 200000.0 | 1.0 | Yes |
| 83 | 1084 | 40.0 | NaN | 690.0 | 180000.0 | 0.0 | No |
| 91 | 1090 | NaN | 19195.0 | NaN | NaN | -1.0 | YES |
| 92 | 1091 | 0.0 | 139619.0 | NaN | NaN | -1.0 | Yes |
| 93 | 1092 | NaN | NaN | NaN | NaN | NaN | YES |
| 94 | 1093 | NaN | NaN | NaN | 550810.0 | NaN | No |
| 95 | 1094 | NaN | NaN | 383.0 | NaN | NaN | No |
| 96 | 1095 | NaN | NaN | NaN | NaN | NaN | No |
| 97 | 1096 | NaN | 45627.0 | NaN | NaN | 6.0 | NO |
| 98 | 1097 | NaN | NaN | NaN | NaN | NaN | NO |
| 99 | 1098 | NaN | 77228.0 | 820.0 | NaN | NaN | Yes |

**2. Null Data**

Some values were explicitly marked as NULL.

Examples:

- Age = NULL
- Monthly_Income = NULL

### 3. Inconsistent Data

Some records contain invalid or inconsistent formats.

Examples:

- Loan_Approved = YES, yes, NO
- Negative age values
- Negative income values

```
    Customer_ID  Age  Monthly_Income  Credit_Score  Loan_Amount  \
87         1088  5.0        -10000.0         200.0     -50000.0

    Previous_Loans Loan_Approved
87            -1.0            No
    Customer_ID  Age  Monthly_Income  Credit_Score  Loan_Amount  \
87         1088  5.0        -10000.0         200.0     -50000.0
91         1090  NaN         19195.0           NaN          NaN
92         1091  0.0        139619.0           NaN          NaN

    Previous_Loans Loan_Approved
87            -1.0            No
91            -1.0           YES
92            -1.0           Yes
    Customer_ID   Age  Monthly_Income  Credit_Score  Loan_Amount  \
84         1085 -15.0         60000.0         730.0     220000.0
85         1086  29.0         42000.0         710.0     130000.0
91         1090   NaN         19195.0           NaN          NaN
93         1092   NaN             NaN           NaN          NaN
97         1096   NaN         45627.0           NaN          NaN
98         1097   NaN             NaN           NaN          NaN

    Previous_Loans Loan_Approved
84             2.0           YES
85             1.0           yes
91            -1.0           YES
93             NaN           YES
97             6.0            NO
98             NaN            NO
```

### 4. Duplicate Records

Some rows were intentionally duplicated to simulate repeated entries in real datasets.

Examples:

- Duplicate customer records with identical values

```
    Customer_ID   Age  Monthly_Income  Credit_Score  Loan_Amount  \
88         1011  56.0         63427.0         735.0     352698.0
89         1026  28.0         45033.0         631.0     465277.0
90         1041  33.0        118447.0         709.0     259186.0

    Previous_Loans Loan_Approved
88             1.0          Yes
89             5.0           No
90             5.0          Yes
```

```
    Customer_ID   Age  Monthly_Income  Credit_Score  Loan_Amount  \
88         1011  56.0         63427.0         735.0     352698.0
89         1026  28.0         45033.0         631.0     465277.0
90         1041  33.0        118447.0         709.0     259186.0

    Previous_Loans Loan_Approved
88             1.0          Yes
89             5.0           No
90             5.0          Yes
```

### 5. Noisy/Outlier Data

Extreme or unrealistic values were added.

```
    Customer_ID    Age  Monthly_Income  Credit_Score  Loan_Amount  \
84         1085  -15.0         60000.0         730.0     220000.0
86         1087  150.0       9999999.0         999.0   99999999.0
87         1088    5.0        -10000.0         200.0     -50000.0
92         1091    0.0        139619.0           NaN          NaN

    Previous_Loans Loan_Approved
84             2.0          YES
86            20.0          Yes
87            -1.0           No
92            -1.0          Yes
    Customer_ID    Age  Monthly_Income  Credit_Score  Loan_Amount  \
86         1087  150.0       9999999.0         999.0   99999999.0
87         1088    5.0        -10000.0         200.0     -50000.0

    Previous_Loans Loan_Approved
86            20.0          Yes
87            -1.0           No
```

**6. Binary Class Labe**

```
Loan_Approved
No     49
Yes    45
YES     3
NO      2
yes     1
Name: count, dtype: int64
```

## Apply Suitable Data Cleaning Techniques

To improve the quality of the dataset, several data cleaning techniques were applied. These techniques ensured that the dataset became more accurate, consistent, and suitable for further analysis and machine learning tasks.

**1. Handling Missing Values**

The dataset contained several missing values in numerical columns such as:

- Age
- Monthly_Income
- Credit_Score
- Loan_Amount
- Previous_Loans

To handle these missing values, the mean value of each respective column was used as a replacement. This method helps preserve the overall distribution of the data while avoiding data loss.

**Technique Used:**

- Mean Imputation

**Example:**

df["Age"].fillna(df["Age"].mean(), inplace=True)

After applying the technique, all missing values were successfully removed from the dataset.

```
Customer_ID       0
Age               0
Monthly_Income    0
Credit_Score      0
Loan_Amount       0
Previous_Loans    0
Loan_Approved     0
dtype: int64
```

**2. Fixing Inconsistent Data**

Some columns contained inconsistent or invalid data entries.

**Problems Identified:**

- Different formats of class labels:
    - YES
    - yes
    - NO
    - Yes
    - No
- Invalid age values below 18 or above 100
- Invalid credit scores below 300 or above 850
- Negative values in income, loan amount, and previous loans

**Solutions Applied:**

**Standardizing Class Labels**

All loan approval labels were converted into a consistent format:

- YES → Yes
- yes → Yes
- NO → No

**Example:**

```
df["Loan_Approved"] = df["Loan_Approved"].replace({
   "YES": "Yes",
   "yes": "Yes",
   "NO": "No"
```

})

```
Loan_Approved
No     51
Yes    49
Name: count, dtype: int64
```

**Correcting Invalid Numerical Values**

Invalid ages and credit scores were replaced using median values because the median is less affected by outliers.

**Example:**

df.loc[(df["Age"] < 18) | (df["Age"] > 100), "Age"] = median_age

Negative values in income and loan-related columns were also replaced with median values.

**3. Removing Duplicate Records**

The dataset contained duplicate rows that could negatively affect data analysis and machine learning model performance.

**Technique Used:**

- Duplicate Removal using **drop_duplicates()**

**Example:**

df = df.drop_duplicates()

After cleaning, duplicate records were completely removed.

**3.4 Treating Noisy and Outlier Values**

The dataset contained noisy and extreme values (outliers) in numerical columns such as:

- Age
- Monthly_Income
- Credit_Score
- Loan_Amount

- Previous_Loans

**Technique Used:**

- Interquartile Range (IQR) Method

The IQR method was used to identify outliers. Values outside the lower and upper bounds were considered outliers and replaced with the median value of the column.

**Formula Used:**

- IQR = Q3 − Q1
- Lower Bound = Q1 − 1.5 × IQR
- Upper Bound = Q3 + 1.5 × IQR

**Example:**

df.loc[(df[col] < lower) | (df[col] > upper), col] = median

This process reduced noise and improved the overall consistency of the dataset.

| | Customer_ID | Age | Monthly_Income | Credit_Score | Loan_Amount |
|---|---|---|---|---|---|
| count | 97.000000 | 97.000000 | 97.000000 | 97.000000 | 97.000000 |
| mean | 1049.092784 | 38.581214 | 79607.606695 | 705.883383 | 265336.298969 |
| std | 28.293139 | 10.194023 | 39640.499378 | 81.326576 | 120046.536045 |
| min | 1001.000000 | 21.000000 | 19195.000000 | 563.000000 | 51701.000000 |
| 25% | 1025.000000 | 29.000000 | 46417.000000 | 641.000000 | 162886.000000 |
| 50% | 1049.000000 | 38.455556 | 77923.000000 | 700.268817 | 263078.000000 |
| 75% | 1073.000000 | 47.000000 | 103504.000000 | 766.000000 | 352698.000000 |
| max | 1098.000000 | 60.000000 | 177829.978495 | 848.000000 | 550810.000000 |

| | Previous_Loans |
|---|---|
| count | 97.000000 |
| mean | 2.217592 |
| std | 1.655631 |
| min | 0.000000 |
| 25% | 1.000000 |
| 50% | 2.000000 |
| 75% | 4.000000 |
| max | 6.000000 |

## Final Dataset Preparation

After cleaning:

- All missing values were handled
- Inconsistent data was corrected
- Duplicate rows were removed
- Outliers and noisy data were treated
- Numerical columns were converted into integer format

Finally, the cleaned dataset was saved as:

Cleaned_Dataset.csv

Link:

Google Colab Link

Uncleaned dataset

Cleaned  dataset